

A BEGINNER'S GUIDE

Amazon Bedrock AgentCore Identity

Securing AI agents, made simple — identity and access explained for everyone.

The Problem, In Plain Terms

An AI agent is a program that does tasks for you — like reading your calendar or updating a document.

To do that, it often needs to connect to other apps (**Google, Slack, GitHub**) and to AWS.

Two simple questions come up

1. How does an app know it's really allowed to talk to the agent?
2. How does the agent safely use your account on another service — without seeing everyone else's?

The three players



The Agent

Does the task for you



Other Apps

Google, Slack, GitHub, AWS



You

The person it acts for

What Is AgentCore Identity?

A managed AWS service that takes care of identity and credentials for AI agents — so builders don't have to create it themselves. Think of it as a secure ID and keychain system for your agents.



Its Own ID

Each agent gets a unique identity — no sharing passwords or accounts.



A Secure Keychain

Safely stores the keys an agent needs to reach other services.



Kept Separate & Safe

Each user's access is isolated, so data never leaks between people.

Why It Matters

Without it, teams spend months building their own login systems, secret stores and audit logs — and it's easy to get security wrong. AgentCore Identity gives you these safely, out of the box.



Save Time

No need to build custom login, secret storage and audit code from scratch.



Safer by Default

Encryption, per-user isolation and an audit trail are built in.



Works With Your Tools

Fits the identity providers and apps your team already uses.

Where You'd Use AgentCore Identity

Everyday jobs where AgentCore Identity handles the sign-in, keys, and permissions safely.



Plan your day

An agent reads your Google Drive or Calendar to help organize your schedule.



Manage GitHub

An agent opens or updates GitHub issues for a developer.



Post to Slack

An agent shares an update in your Slack workspace.



Nightly data job

A behind-the-scenes job reads a shared Salesforce report each night.

Works With What You Already Have

Sign in with

Common identity providers

Amazon Cognito

Okta

Microsoft Entra ID

Google

Connect to

Ready-made app setups

GitHub

Google

Slack

Salesforce

Microsoft

You generally don't have to build these connections from scratch.

The Main Building Blocks

Four parts work together. Each one has a simple real-world analogy.



Identity Directory

The agent's ID card

Lists each agent and who it is.



Agent Authorizer

The doorman

Checks whether a caller is allowed in.



Resource Credential Providers

The keyring for each service

Knows how to get keys for Google, GitHub, Slack, and more.



Token Vault

The safe

Stores those keys securely, locked and separated per user.

Two Directions: In and Out

One easy idea makes the whole system simple to reason about: access flows in two directions.



Inbound

Coming IN to the agent

Who is allowed to call the agent? Think of it as the front-door check that decides who gets in.



Outbound

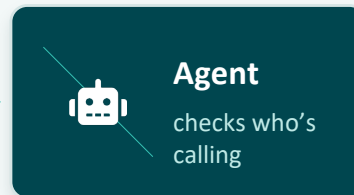
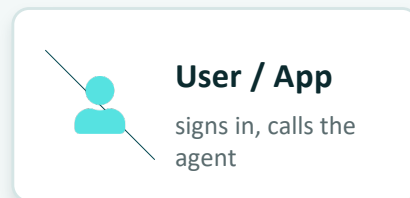
Agent going OUT to other apps

What can the agent reach, and with whose permission? Think of it as the agent using a key to open another app.

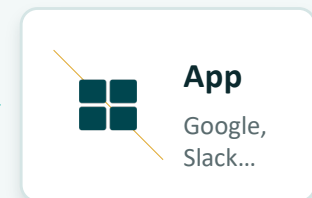
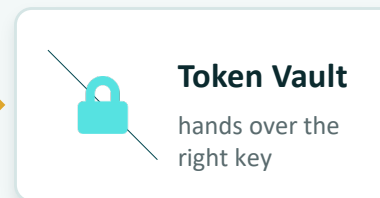
The Flow: Inbound In, Outbound Out

One picture ties it together: a request comes IN to the agent, then the agent reaches OUT to do the work.

INBOUND



OUTBOUND



"Who is allowed to call the agent?"

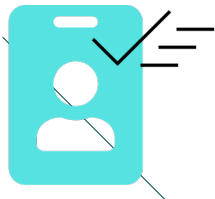
"What can the agent reach, and with whose key?"

Inbound: The Front Door

Inbound auth answers one question: **who is allowed to call the agent?** Two ways to check at the door.



Real-life analogy: a building entrance — either a staff badge that the door reader trusts, or a visitor pass signed by reception.



IAM (a staff badge)

The default for callers already inside AWS — other AWS services or functions. The door reader trusts the badge automatically. No extra setup.



Sign-in token (a visitor pass)

For people signing in through a login provider (Cognito, Okta, Microsoft). They arrive with a signed pass the agent checks before letting them in.

Inbound: When to Use IAM vs. JWT

This is the front-door choice: how does the agent check who is calling it? Pick based on WHO the caller is.



Use IAM (SigV4)

when the caller is inside AWS

Other AWS services, Lambda functions, or a Gateway calling the agent.

Analogy: a staff badge the door reader already trusts — no setup.



Use a JWT token

when a person signs in

End users logging in through Cognito, Okta, or Microsoft Entra ID.

Analogy: a signed visitor pass the agent checks at the door.

Rule of thumb: caller is an AWS service → IAM. Caller is a signed-in person → JWT.

Outbound: The Agent's Keys

Outbound auth answers: what can the agent reach, and with what key? The right key depends on the lock.



Real-life analogy: a keyring — some doors take a master key you own (IAM), some take a spare key a friend lends you (OAuth), some use biometric or face detection to open the door (API key).



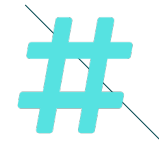
IAM — your master key

For AWS's own services. The agent already owns the key; AWS recognizes it directly. Best for reaching things inside AWS.



OAuth — a borrowed key

For apps like Google or Slack. The service lends a key after someone approves — kept safely in the vault and reused.



API key — a gate code

For simpler services that just want a secret code. Stored safely and sent with each request — no login screen involved.

Outbound: When to Use IAM vs. API Keys

Two ways an agent can carry credentials outbound. A simple rule: is the thing you're reaching inside AWS, or an outside service with a secret code?



Use IAM

for AWS's own services

When the agent reaches AWS services (S3, DynamoDB, Lambda).

Analogy: your own master key — AWS already recognizes it, nothing to hand out.



Use an API Key

for simple outside services

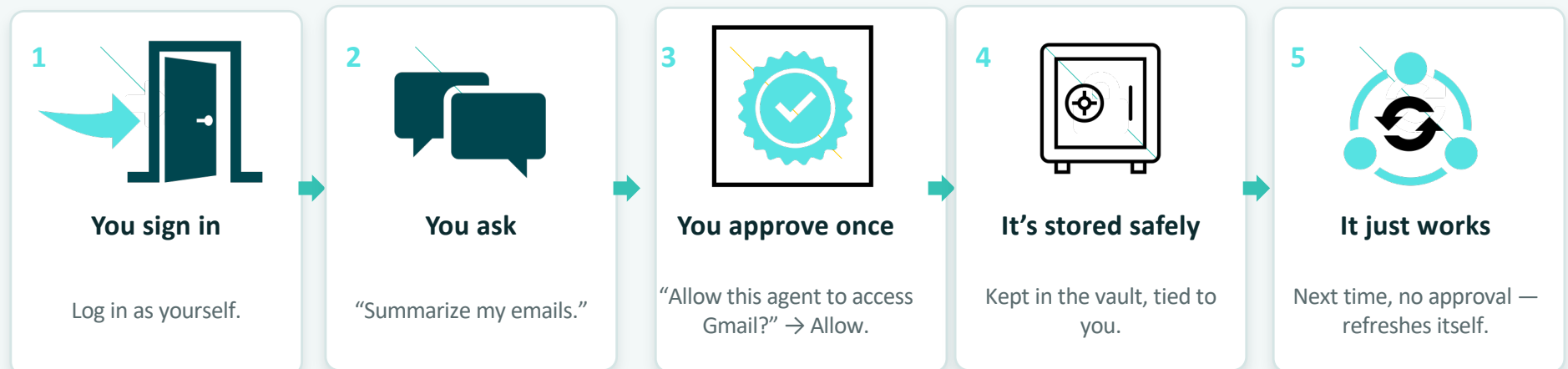
When a third-party service just wants a secret code (no login screen).

Analogy: a gate code — stored safely in the vault and sent with each request.

Rule of thumb: inside AWS → IAM. Outside service with a secret code → API key. (Needs a user's permission → OAuth.)

How It Works: A Simple Example

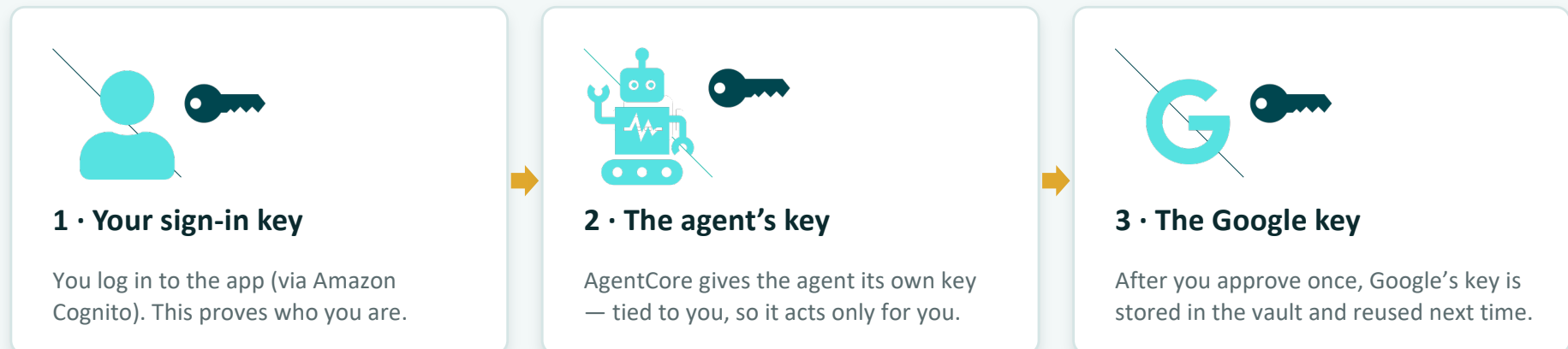
The most common case: an agent doing something for a specific person.



Approve once → then it just works.

Behind the Scenes: Three Keys

A real example from AWS: “Schedule a meeting in my Google Calendar.” Three keys are handed off — the user only ever sees the first sign-in.



Powered by Amazon Cognito · keys are bound together so one agent, one user, stays isolated.

Where Credentials Live

Different secrets are kept in different safe places — here's the simple map of what goes where, and how each is protected.

Token Vault

encrypted with AWS KMS

- **User access tokens**
(~1 hour)
- **User refresh tokens**
(~30 days, auto-refresh)

Encrypted per user · customer-managed key recommended



AWS Secrets Manager

app-level secrets

- **API keys**
- **OAuth client secrets**
(the app's own id & secret)

Shared across users, not per-person

The agent's own workload token is short-lived (~1 hour) and not stored long term.

The Other Pattern: Background Jobs

Not everything is done for a specific person. Sometimes an agent just runs on its own.



For a person

Approve once

The person allows it the first time, then the agent acts on their behalf — seeing only their data.



Background job

Agent's own identity

A nightly job that reads a shared report needs no "Allow" screen — it uses its own identity and keys.

Two Kinds of OAuth: 2-Legged & 3-Legged

“Legs” just means how many parties are involved in handing over the key.

2-Legged (no person)

Agent ↔ Service

Two parties: the agent and the service. The agent uses its own identity — no human approves anything.

Analogy: a vending machine you own — it restocks itself with its own key, nobody signs in.

3-Legged (on your behalf)

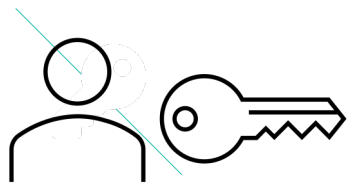
You ↔ Agent ↔ Service

Three parties: you, the agent, and the service. You approve once, then the agent acts as you.

Analogy: a valet key you hand over — they drive your car, but only within the limits you allowed.

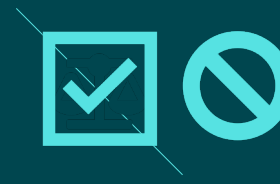
Knowing Who ≠ Knowing What They Can Do

So far we've made sure the agent knows WHO is involved and holds the right keys. But proving identity isn't the same as deciding what's allowed — that's a separate question.



So far: Identity & Keys

The agent can prove who it is and who it acts for, and it holds the right keys to reach a service.



Next: What's Allowed

Even a trusted agent shouldn't do everything. We need rules for which actions are permitted — that's authorization.

Analogy: having a key to the building doesn't mean you're allowed into every room. Next, we set the house rules.

Setting the Rules: Cedar Policy

Cedar is an open-source “**rulebook**” language from AWS for writing “**who can do what.**” Think of a club bouncer: Cedar is the rulebook they follow.

Every rule has four parts

Principal	Who is asking (a person or agent)
Action	What they want to do
Resource	What they want to use
When	Any conditions that must be true

permit = green light (allow)

forbid = red light (always wins)

If no rule matches, the answer is “no” by default.



Bouncer analogy:

permit lets you in, forbid turns you away, and “when” is “only if you have the right wristband.”

What Is the AgentCore Policy Engine?

If Cedar is the rulebook, the **Policy Engine** is the referee that actually enforces it — a built-in AWS service that decides whether each thing an agent tries to do is allowed.



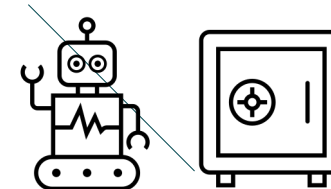
The rule-enforcer

A managed service that reads your Cedar rules and applies them to what the agent tries to do.



Lives at the gateway

It sits at the AgentCore Gateway — the checkpoint every tool request passes through.



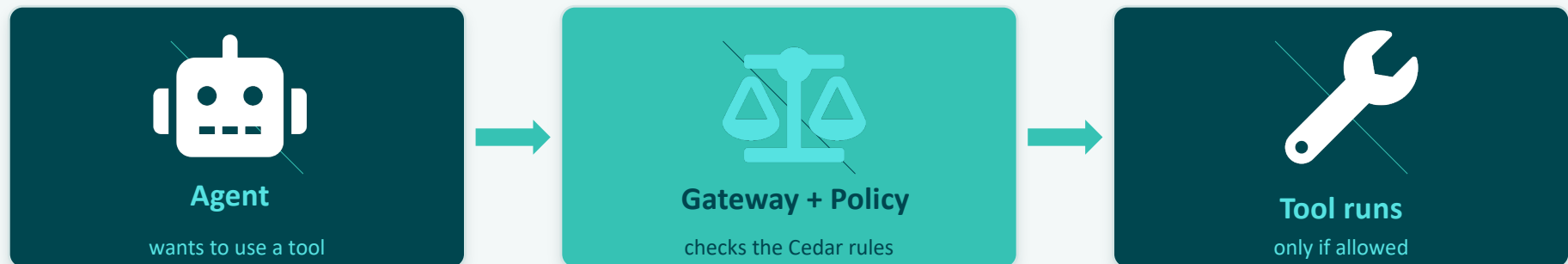
Outside the agent

Because it runs apart from the agent's code, a tricked or buggy agent still can't get around it.

In one line: it's the managed "yes/no" checkpoint for agent actions — next, let's see how a request flows through it.

How the Policy Engine Works

The Policy Engine applies Cedar rules to every tool the agent tries to use — checked automatically, before anything runs, and from outside the agent's own code.



Why outside the agent? Even if the agent is confused or tricked, it can't break a rule it isn't allowed to break — the guardrail lives outside its reasoning. You can also write rules in plain English and let AgentCore turn them into Cedar.

A Real Cedar Policy

Rules read almost like plain English. Here is one that allows an agent to process refunds — but only small ones.

```
permit(  
  principal is refund-agent,  
  action   == "process_refund",  
  resource == refund-gateway  
)  
when {  
  context.input.amount < 500  
};
```

permit	Allow this — a green light
principal	WHO: the refund agent
action	WHAT: process a refund
resource	WHERE: the refund gateway
when	ONLY IF: amount is under \$500

A \$600 refund? Automatically denied.

Who Asked for What: The Audit Trail

Every time an agent acts, AgentCore Identity writes it down — like a building’s visitor logbook and security cameras, but for agent actions.

Each entry captures

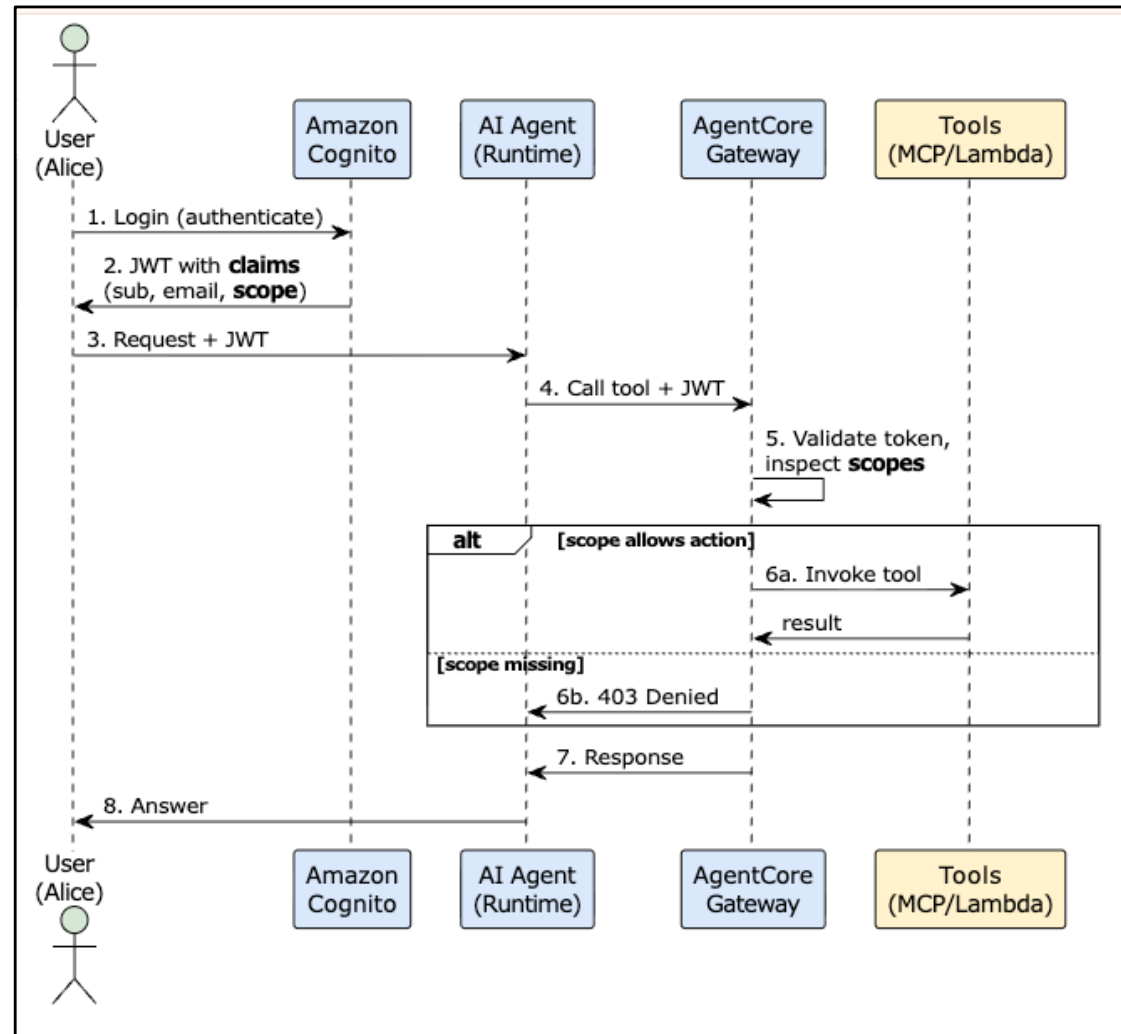
-  **WHO** — which user asked
-  **WHICH AGENT** — acted on their behalf
-  **WHAT** — the action or tool used
-  **WHEN** — the date and time
-  **RESULT** — allowed or denied



Like a logbook

The records stream to **Amazon CloudWatch**, giving one tamper-resistant history you can search later.

Great for trust, troubleshooting & compliance.



KEY TAKEAWAYS

Approve once — then it just works



Agents get their own safe identity

No shared passwords or accounts.



Everyone's access stays separate

One person's data never leaks to another.



Sign-in & keys are handled for you

Approve once; it refreshes automatically.

Learn more: [AWS Documentation](#) — [Amazon Bedrock AgentCore Identity & Blog](#)